

Challenge: Property Sales Conversational Agent

Part 1: Business Goal & expected task from the AI assistant

Business goal:

You are building a conversational AI assistant for a property developer website. The name of the property firm is Silver Land Properties & the AI assistant is working on behalf of this firm. The goal for the property sales AI assistant is to greet the enquiring user, understand their property purchase preferences, recommend the best-match property from the available property database and nudge the user to schedule a property viewing. It needs to be smart enough to recommend/cross-sell the right property option closest to the buyer criteria if there are no direct matches available in the database.

Expected tasks:

When a new property buyer lands on the site and opens the chat, the assistant should:

1. Understand and probe property buyer preferences

- Greet buyer and ask proactive questions to clarify:
 - Preferred location (city)
 - Budget range
 - Unit size (e.g. 1-bed, 2-bed)

2. Recommend suitable properties

- Search within a simple property projects database provided.
- Shortlist and present 1–3 relevant projects with:
 - Project name
 - City / location
 - Starting price / price band
 - Key features (e.g. amenities, unit mix, pricing)

3. Answer questions and address objections

- The assistant should be able to answer typical questions about each project by reading from project descriptions or invoke web search tool (optional) as necessary.
- Examples:
 - “What are the project amenities?”
 - “What is the completion date?”
 - “Do you have 3-bed units under X budget?”
- If information is not available, the agent should say so explicitly.

4. Drive towards a property visit booking (Primary business goal)

- Once the user seems interested in a project, the agent should:
 - Confirm project name
 - Propose booking a property visit
 - Collect lead details:
 - Name
 - Email
 - Chosen project name
 - City
- Store each confirmed booking as a row in a separate “visit_bookings” table.

5. Basic product expectations

- Interactions should feel like a single conversation, not separate APIs.
- The agent should avoid hallucinating details that are not present in the DB or brochures. If unsure, it should admit that clearly.

The goal is not to build a full production system, but to demonstrate a sensible design, correct use of tools, and an organized orchestration approach.

Part 2: Technical Scope and Constraints

Design and implement a minimal backend that exposes the conversational agent over a REST API and uses a tool-based agent orchestration pattern.

2.1 Architecture (Core Expectations)

- Language / Stack
 - Python backend using a modern web framework:
 - Preferably Django Ninja with Ninja Extra with OOP controllers.
- Agent Orchestration
 - Use LangGraph to model the conversational agent and its tools. The workflow must be able to process:
 - Run a T2SQL queries over the database via Vanna
 - Invoke web search about a project specific query for which information is not available in the provided DB. example, ‘what are the schools near project name X?’
 - Synthesize the response for the user
- Databases: Use SQLite (or PostgreSQL if you prefer) as the relational DB.
 - Minimum tables:
 - **projects** (or equivalent) to store property projects and metadata.
 - **bookings** to store the final booked leads.
 - **leads** to store the lead information throughout the conversation

- The lead information is composed of
 - First Name
 - Last Name
 - Email
 - Preferences (e.g. property type, bedrooms, price range, etc)
- LLM & embedding models of your choice. Share rationale if you prefer specific models.

2.2 Data Access Tools

1. Text-to-SQL tool (over the projects DB)

Core requirement: the agent must be able to translate a natural language query into a SQL query and execute it against your relational DB.

 - Preferred: Use Vanna with ChromaDB to power text-to-SQL as in the reference spec.
 - If you are short on time or unfamiliar with Vanna, you may implement a simpler approach, but your graph/agent framework should still treat this as a distinct “SQL tool” called by the agent.
2. Web search for any project specific information (Optional but has extra points): run web search tool as a fall back option when user asks a specific query about a given project. Do not invoke web search when the user is placing a broad criteria and asking for project recommendations. Projects recommended should be strictly within the database.

2.3 API Design

- Expose at least two REST endpoint:
 - **POST /conversations**
 - **POST /agents/chat**
 - Request body: { "message": "...", "conversation_id": "UUID" }
 - Response: the agent's reply plus any structured data you think is useful (e.g. shortlisted project IDs).
- Keep your routes REST-like and use appropriate HTTP status codes.
- Authentication is optional if you have time.

3. Minimum deliverables

1. Running backend service
2. LangGraph-based agent or other framework (specify)
3. Ability for the AI assistant to drive the user towards the goal (property viewing)
4. Basic documentation
5. Deployment on Render or Vercel

4. How we'll evaluate your submission

- How you model the graph and tools in LangGraph.
- How clean and modular your Python code and directory structure are.
- How deep and structured your tests are.
- Whether the agent maintains coherent conversation flow, correctly routes and fetches relevant data, drives the user towards the goal.